

DAY 7

Backend Preparation

Project: Customer Churn Prediction & LTV Engine

Role: Member 3 | Date: 19-05-2026

Objective

Build the foundational backend Python files that will power the Customer Churn Prediction API in Week 3. This includes a health check endpoint, input schema definition, data validation functions, automated unit tests, and API architecture notes.

Why Backend Preparation?

In Week 3 we will build a real API (using FastAPI) that takes customer data and returns a churn prediction. To avoid doing everything at once, we prepare the structure and logic today — like building the skeleton of a house before the walls go up.

Folder Structure Created

Folder	Purpose
backend/api/	API endpoint functions
backend/utils/	Helper / utility functions
tests/unit/	Automated tests for utilities
docs/api_docs/	API architecture documentation

Files Created

FILE 1 — health_check.py

backend/api/health_check.py

Purpose: Checks if the API server is alive. Like a 'ping' for the server.

Code:

```
def health_check():
    return {'status': 'ok', 'message': 'API is running', 'version': '1.0.0'}

def detailed_health():
    return {'status': 'ok', 'model_loaded': False, 'db_connected': False}

if __name__ == '__main__':
    print(health_check())
```

How to run in terminal:

```
python backend/api/health_check.py
```

Expected output: {'status': 'ok', 'message': 'API is running', 'version': '1.0.0'}

FILE 2 — prediction_schema.py

backend/api/prediction_schema.py

Purpose: Defines WHAT data the API expects. Think of it as a form template.

Code:

```
CUSTOMER_SCHEMA = {
    'tenure': {'type': 'integer', 'example': '12'},
    'MonthlyCharges': {'type': 'float', 'example': '65.50'},
    'Contract': {'type': 'string', 'example': 'Month-to-month'},
    'InternetService': {'type': 'string', 'example': 'DSL'},
}

REQUIRED_FIELDS = ['tenure', 'MonthlyCharges', 'Contract', 'InternetService']
```

How to run in terminal:

```
python backend/api/prediction_schema.py
```

Expected output: tenure | integer | REQUIRED MonthlyCharges | float | REQUIRED ...

FILE 3 — data_validator.py

backend/utls/data_validator.py

Purpose: Checks incoming data is valid BEFORE making a prediction. Catches bad input early.

Code:

```
def validate_tenure(value):
    if not isinstance(value, (int, float)):
        return False, 'Tenure must be a number'
    if value < 0: return False, 'Cannot be negative'
    if value > 72: return False, 'Cannot exceed 72'
    return True, 'Valid'

def validate_contract(value):
    valid = ['Month-to-month', 'One year', 'Two year']
    if value not in valid: return False, f'Must be one of {valid}'
    return True, 'Valid'
```

How to run in terminal:

```
python backend/utils/data_validator.py
```

Expected output: (True, 'Valid') (False, 'Cannot be negative')

FILE 4 — test_data_validator.py

```
tests/unit/test_data_validator.py
```

Purpose: Automated tests that confirm the validator works correctly every time.

Code:

```
import sys; sys.path.append('.')
from backend.utils.data_validator import validate_tenure, validate_contract

print(validate_tenure(12)) # (True, 'Valid')
print(validate_tenure(-5)) # (False, 'Cannot be negative')
print(validate_contract('One year')) # (True, 'Valid')
print(validate_contract('Weekly')) # (False, ...)
```

How to run in terminal:

```
python tests/unit/test_data_validator.py
```

Expected output: All True/False results printed — confirms validator logic works.

FILE 5 — api_structure_notes.txt

```
docs/api_docs/api_structure_notes.txt
```

A plain-text architecture document describing the planned API endpoints, input/output formats, files created today, and tools planned for Week 3 (FastAPI, Pydantic, Uvicorn).

NOTE: Always use forward slashes (/) not backslashes when running Python files in the VS Code terminal. Example: `python backend/api/health_check.py`

Day 7 Deliverables

File	Location	Status
health_check.py	backend/api/	Done
prediction_schema.py	backend/api/	Done
data_validator.py	backend/utis/	Done
test_data_validator.py	tests/unit/	Done
api_structure_notes.txt	docs/api_docs/	Done